

Express + MongoDB Task Manager API with Next.js UI (Bootstrap)

A complete, minimal-but-solid starter that gives you:

- **Express REST API** (CRUD) with **MongoDB/Mongoose**
- **Search** by keyword (title/description)
- **Filter** by status (`pending`, `in-progress`, `completed`)
- **Next.js UI** (App Router) that consumes the Express API
- **Bootstrap 5** for styling

Project layout (monorepo):

```
.
├── server/           # Express + MongoDB API
└── client/          # Next.js 14 UI with Bootstrap
```

1) Server — Express + MongoDB

server/package.json

```
{
  "name": "task-api-server",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "nodemon src/index.js",
    "start": "node src/index.js"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.19.2",
    "mongoose": "^8.4.1",
    "morgan": "^1.10.0"
  },
  "devDependencies": {
    "nodemon": "^3.1.0"
  }
}
```

server/.env.example

```
PORT=4000
MONGODB_URI=mongodb://127.0.0.1:27017/task_manager_db
```

server/src/index.js

```
import 'dotenv/config';
import express from 'express';
import mongoose from 'mongoose';
import morgan from 'morgan';
import cors from 'cors';
import tasksRouter from './tasks.routes.js';

const app = express();
const PORT = process.env.PORT || 4000;

// Middleware
app.use(cors({ origin: true, credentials: true }));
app.use(express.json());
app.use(morgan('dev'));

// Health
app.get('/api/health', (req, res) => res.json({ ok: true, time: new
Date().toISOString() }));

// Routes
app.use('/api/tasks', tasksRouter);

// 404
app.use((req, res) => res.status(404).json({ message: 'Not Found' }));

// DB + start
const MONGODB_URI = process.env.MONGODB_URI || 'mongodb://127.0.0.1:27017/
task_manager_db';

mongoose
  .connect(MONGODB_URI)
  .then(() => {
    console.log('✅ MongoDB connected');
    app.listen(PORT, () => console.log(`✅ API listening on http://localhost:$
{PORT}`));
  })
  .catch((err) => {
    console.error('MongoDB connection error:', err);
  });
```

```
    process.exit(1);
  });
```

server/src/task.model.js

```
import mongoose from 'mongoose';

const TaskSchema = new mongoose.Schema(
  {
    title: { type: String, required: true, trim: true },
    description: { type: String, default: '' },
    status: {
      type: String,
      enum: ['pending', 'in-progress', 'completed'],
      default: 'pending',
      index: true,
    },
  },
  { timestamps: true }
);

// for case-insensitive text search on title/description
TaskSchema.index({ title: 'text', description: 'text' });

export default mongoose.model('Task', TaskSchema);
```

server/src/tasks.routes.js

```
import { Router } from 'express';
import Task from './task.model.js';

const router = Router();

// CREATE
router.post('/', async (req, res) => {
  try {
    const { title, description = '', status = 'pending' } = req.body;
    const task = await Task.create({ title, description, status });
    res.status(201).json(task);
  } catch (err) {
    res.status(400).json({ message: 'Failed to create task', error:
err.message });
  }
});
```

```

// READ (list with search & filter)
// GET /api/tasks?q=keyword&status=pending&page=1&limit=10
router.get('/', async (req, res) => {
  try {
    const { q, status, page = 1, limit = 10 } = req.query;
    const filters = {};

    if (status && ['pending', 'in-progress', 'completed'].includes(status)) {
      filters.status = status;
    }

    let query = Task.find(filters);

    if (q && q.trim()) {
      // text search
      query = query.find({ $text: { $search: q.trim() } });
    }

    const pageNum = Math.max(parseInt(page, 10) || 1, 1);
    const limitNum = Math.min(Math.max(parseInt(limit, 10) || 10, 1), 100);

    const [items, total] = await Promise.all([
      query
        .sort({ createdAt: -1 })
        .skip((pageNum - 1) * limitNum)
        .limit(limitNum),
      Task.countDocuments(q ? { ...filters, $text: { $search: q.trim() } } :
filters),
    ]);

    res.json({ items, total, page: pageNum, limit: limitNum });
  } catch (err) {
    res.status(500).json({ message: 'Failed to fetch tasks', error:
err.message });
  }
});

// READ (by id)
router.get('/:id', async (req, res) => {
  try {
    const task = await Task.findById(req.params.id);
    if (!task) return res.status(404).json({ message: 'Task not found' });
    res.json(task);
  } catch (err) {
    res.status(400).json({ message: 'Invalid id', error: err.message });
  }
});

```

```

// UPDATE
router.put('/:id', async (req, res) => {
  try {
    const { title, description, status } = req.body;
    const task = await Task.findByIdAndUpdate(
      req.params.id,
      { title, description, status },
      { new: true, runValidators: true }
    );
    if (!task) return res.status(404).json({ message: 'Task not found' });
    res.json(task);
  } catch (err) {
    res.status(400).json({ message: 'Failed to update task', error:
err.message });
  }
});

// DELETE
router.delete('/:id', async (req, res) => {
  try {
    const task = await Task.findByIdAndDelete(req.params.id);
    if (!task) return res.status(404).json({ message: 'Task not found' });
    res.json({ message: 'Task deleted' });
  } catch (err) {
    res.status(400).json({ message: 'Failed to delete task', error:
err.message });
  }
});

export default router;

```

Quick API test (optional)

```

# create
curl -X POST http://localhost:4000/api/tasks
  -H 'Content-Type: application/json'
  -d '{"title":"Write docs","description":"Finish README","status":"in-
progress"}'

# list with search & filter
curl 'http://localhost:4000/api/tasks?q=write&status=in-progress'

```

2) Client — Next.js 14 (App Router) + Bootstrap

client/package.json

```
{
  "name": "task-ui-client",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start"
  },
  "dependencies": {
    "bootstrap": "^5.3.3",
    "next": "14.2.4",
    "react": "18.3.1",
    "react-dom": "18.3.1"
  }
}
```

client/.env.local.example

```
# Point this to your Express API base URL
NEXT_PUBLIC_API_URL=http://localhost:4000/api
```

client/next.config.js

```
/** @type {import('next').NextConfig} */
const nextConfig = {};
export default nextConfig;
```

client/src/app/layout.js

```
import 'bootstrap/dist/css/bootstrap.min.css';
import { useEffect } from 'react';

export const metadata = { title: 'Task Manager', description:
'Express + MongoDB + Next.js' };

export default function RootLayout({ children }) {
  // Load Bootstrap JS on client
  useEffect(() => {
```

```

import('bootstrap/dist/js/bootstrap.bundle.min.js');
}, []);

return (
  <html lang="en">
    <body className="bg-light">{children}</body>
  </html>
);
}

```

client/src/app/page.js

```

'use client';
import { useEffect, useMemo, useState } from 'react';

const API = process.env.NEXT_PUBLIC_API_URL;
const STATUSES = ['pending', 'in-progress', 'completed'];

export default function HomePage() {
  const [tasks, setTasks] = useState([]);
  const [total, setTotal] = useState(0);
  const [loading, setLoading] = useState(false);
  const [q, setQ] = useState('');
  const [status, setStatus] = useState('');
  const [editing, setEditing] = useState(null); // task object or null

  const queryString = useMemo(() => {
    const params = new URLSearchParams();
    if (q) params.set('q', q);
    if (status) params.set('status', status);
    return params.toString();
  }, [q, status]);

  const fetchTasks = async () => {
    setLoading(true);
    try {
      const res = await fetch(`${API}/tasks${queryString ? `?${queryString}` : ''}`);
      const data = await res.json();
      setTasks(data.items ?? data);
      setTotal(data.total ?? (data.items ? data.items.length : 0));
    } catch (e) {
      console.error(e);
      alert('Failed to load tasks');
    } finally {
      setLoading(false);
    }
  };
}

```

```

    }
  };

  useEffect(() => { fetchTasks(); }, [queryString]);

  const onSubmit = async (e) => {
    e.preventDefault();
    const form = e.currentTarget;
    const payload = {
      title: form.title.value,
      description: form.description.value,
      status: form.status.value,
    };

    try {
      const url = editing ? `${API}/tasks/${editing._id}` : `${API}/tasks`;
      const method = editing ? 'PUT' : 'POST';
      const res = await fetch(url, {
        method,
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(payload),
      });
      if (!res.ok) throw new Error('Request failed');
      form.reset();
      setEditing(null);
      await fetchTasks();
      const modalEl = document.getElementById('taskModal');
      const modal = window.bootstrap?.Modal.getInstance(modalEl) || new
window.bootstrap.Modal(modalEl);
      modal.hide();
    } catch (e) {
      console.error(e);
      alert('Save failed');
    }
  };

  const onDelete = async (id) => {
    if (!confirm('Delete this task?')) return;
    try {
      const res = await fetch(`${API}/tasks/${id}`, { method: 'DELETE' });
      if (!res.ok) throw new Error('Delete failed');
      await fetchTasks();
    } catch (e) {
      console.error(e);
      alert('Delete failed');
    }
  };
};

```

```

const openCreate = () => {
  setEditing(null);
  const modal = new
window.bootstrap.Modal(document.getElementById('taskModal'));
  modal.show();
};

const openEdit = (task) => {
  setEditing(task);
  // populate form fields shortly after opening
  setTimeout(() => {
    const form = document.getElementById('taskForm');
    if (form && task) {
      form.title.value = task.title;
      form.description.value = task.description || '';
      form.status.value = task.status;
    }
  }, 0);
  const modal = new
window.bootstrap.Modal(document.getElementById('taskModal'));
  modal.show();
};

return (
  <div className="container py-4">
    <div className="d-flex align-items-center justify-content-between mb-3">
      <h1 className="h3 mb-0">Task Manager</h1>
      <button className="btn btn-primary" onClick={openCreate}>
        + New Task
      </button>
    </div>

    <div className="card shadow-sm">
      <div className="card-body">
        <div className="row g-2 align-items-end">
          <div className="col-sm-6">
            <label className="form-label">Search</label>
            <input
              className="form-control"
              placeholder="Search title or description..."
              value={q}
              onChange={(e) => setQ(e.target.value)}
            />
          </div>
          <div className="col-sm-4">
            <label className="form-label">Status</label>
            <select className="form-select" value={status} onChange={(e) =>
setStatus(e.target.value)}>

```

```

        <option value="">All</option>
        {STATUSES.map((s) => (
          <option key={s} value={s}>{s}</option>
        ))}
      </select>
    </div>
    <div className="col-sm-2 d-grid">
      <button className="btn btn-outline-secondary" onClick={() => {
setQ(''); setStatus(''); }}>
        Reset
      </button>
    </div>
  </div>
</div>

<div className="card shadow-sm mt-3">
  <div className="card-body">
    <div className="d-flex justify-content-between align-items-center
mb-2">
      <div><strong>Total:</strong> {total}</div>
      {loading && <div className="text-muted small">Loading...</div>}
    </div>

    <div className="table-responsive">
      <table className="table table-hover align-middle">
        <thead>
          <tr>
            <th>Title</th>
            <th>Description</th>
            <th>Status</th>
            <th>Created</th>
            <th style={{ width: 140 }}>Actions</th>
          </tr>
        </thead>
        <tbody>
          {tasks.length === 0 && (
            <tr>
              <td colspan={5} className="text-center text-muted">No tasks
found.</td>
            </tr>
          )}
          {tasks.map((t) => (
            <tr key={t._id}>
              <td className="fw-semibold">{t.title}</td>
              <td>{t.description}</td>
              <td>
                <span className={`badge text-bg-${t.status} ==

```

```

'completed' ? 'success' : t.status === 'in-progress' ? 'warning' : 'secondary'}
`>
        {t.status}
        </span>
    </td>
    <td>{new Date(t.createdAt).toLocaleString()}</td>
    <td>
        <div className="btn-group btn-group-sm" role="group">
            <button className="btn btn-outline-primary" onClick={()
=> openEdit(t)}>Edit</button>
            <button className="btn btn-outline-danger" onClick={()
=> onDelete(t._id)}>Delete</button>
        </div>
    </td>
</tr>
    </tbody>
</table>
</div>
</div>
</div>

{ /* Modal */ }
<div className="modal fade" id="taskModal" tabIndex="-1" aria-
hidden="true">
    <div className="modal-dialog">
        <div className="modal-content">
            <div className="modal-header">
                <h5 className="modal-title">{editing ? 'Edit Task' : 'New Task'}</
h5>
                <button type="button" className="btn-close" data-bs-
dismiss="modal" aria-label="Close"></button>
            </div>
            <form id="taskForm" onSubmit={onSubmit}>
                <div className="modal-body">
                    <div className="mb-3">
                        <label className="form-label">Title</label>
                        <input name="title" className="form-control" required />
                    </div>
                    <div className="mb-3">
                        <label className="form-label">Description</label>
                        <textarea name="description" className="form-control" rows={3}
></textarea>
                    </div>
                    <div className="mb-1">
                        <label className="form-label">Status</label>
                        <select name="status" className="form-select"
defaultValue="pending">

```

```

        {STATUSES.map((s) => (
          <option key={s} value={s}>{s}</option>
        ))}
      </select>
    </div>
  </div>
  <div className="modal-footer">
    <button type="button" className="btn btn-outline-secondary"
data-bs-dismiss="modal">Cancel</button>
    <button type="submit" className="btn btn-primary">Save</button>
  </div>
</form>
</div>
</div>
</div>
</div>
);
}

```

3) How to Run (Localhost)

Prereqs

- Node.js 18+
- MongoDB running locally (or Atlas URI)

Steps

1. Server

```

cd server
cp .env.example .env
# update MONGODB_URI if needed
npm install
npm run dev
# API on http://localhost:4000

```

2. Client

```

cd client
cp .env.local.example .env.local
# make sure NEXT_PUBLIC_API_URL=http://localhost:4000/api
npm install

```

```
npm run dev
# UI on http://localhost:3000
```

4) Notes & Extensibility

- **CORS** is enabled broadly for local dev. Tighten in production.
- **Text index** enables keyword search; for large data you may want dedicated indices and pagination UI.
- Add validation or use a schema library (e.g., `zod`) on both client & server.
- Consider adding optimistic UI updates, pagination controls, and status quick-toggles.

5) Bonus: Simple Seed Script (optional)

```
server/src/seed.js
```

```
import 'dotenv/config';
import mongoose from 'mongoose';
import Task from './task.model.js';

const MONGODB_URI = process.env.MONGODB_URI;

async function run() {
  await mongoose.connect(MONGODB_URI);
  await Task.deleteMany({});
  await Task.insertMany([
    { title: 'Write proposal', description: 'Initial draft', status:
'pending' },
    { title: 'Build API', description: 'CRUD + search', status: 'in-progress' },
    { title: 'Polish UI', description: 'Bootstrap styling', status:
'completed' },
  ]);
  console.log('Seeded!');
  await mongoose.disconnect();
}

run().catch((e) => { console.error(e); process.exit(1); });
```

Run it:

```
node src/seed.js
```

That's it — a clean baseline with **CRUD**, **search**, **status filter**, and a **Bootstrap-powered Next.js UI**.